

Ex. No. 4

Study and Implementation of Amazon Toolkit: Sagemaker

Date:

PROBLEM STATEMENT

Use Amazon SageMaker to create new notebooks that upload data, train and tune models, move back and forth between steps to adjust models, compare results, and deploy models.

AIM

To study and implement the Amazon Web Services (AWS) toolkit SageMaker for building, training, and deploying a machine learning model.

PROCEDURE

1. Create a SageMaker notebook instance.
2. Prepare the data.
3. Train the model to learn from the data.
4. Deploy the model.
5. Evaluate the ML model's performance.

AWS SAGEMAKER CONSOLE SETUP

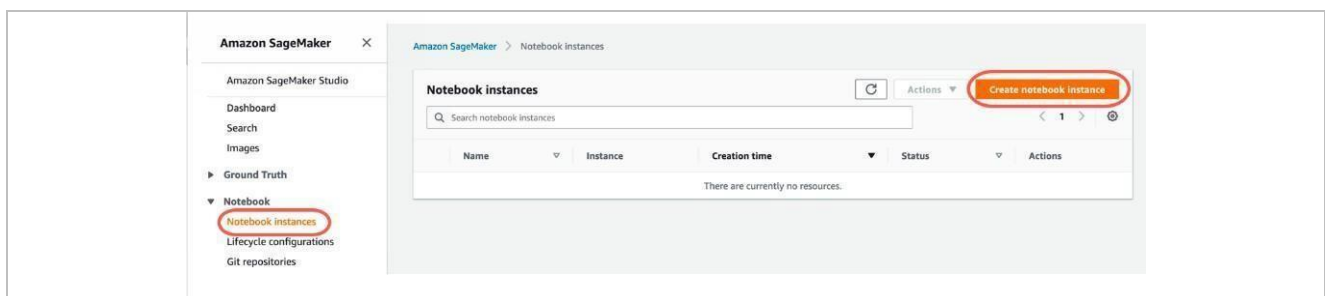
Step 1 — Create an Amazon SageMaker Notebook Instance

In this step, a notebook instance is created to download and process data. As part of the creation process, an IAM role is also created that allows SageMaker to access data in Amazon S3.

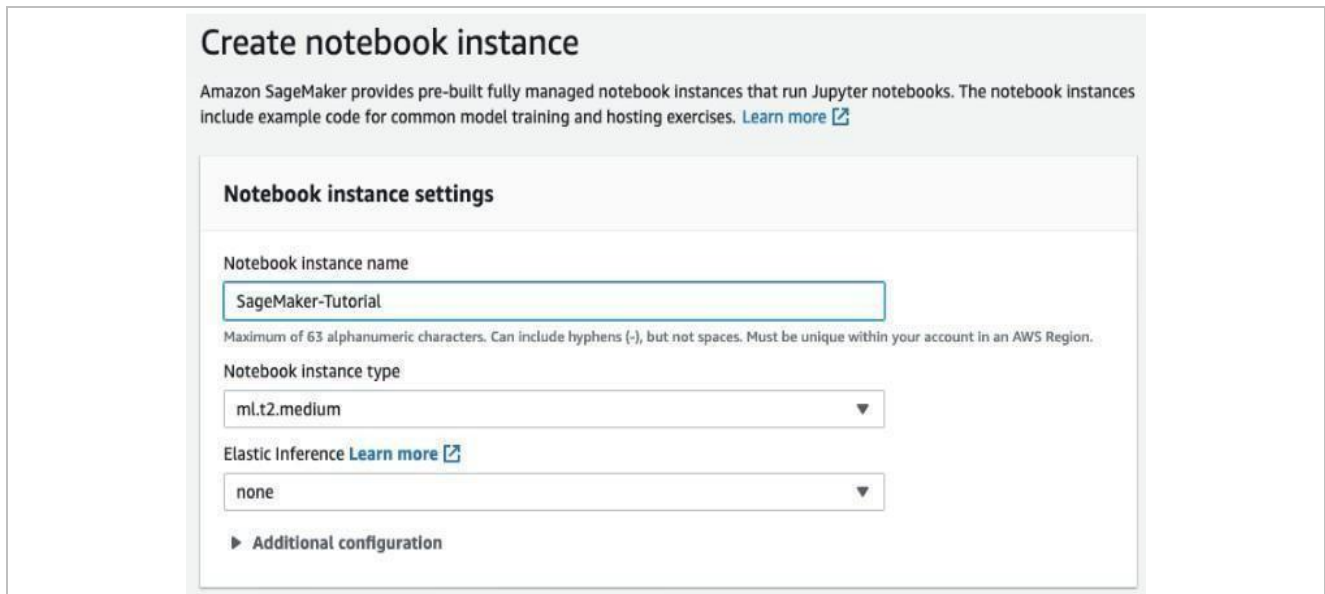
- a. Sign in to the Amazon SageMaker console, and in the top-right corner, select the preferred AWS Region.



- b. In the left navigation pane, choose Notebook instances, then choose Create notebook instance.



- c. Fill in the following fields — Notebook instance name: SageMaker-Tutorial, Notebook instance type: ml.t2.medium, Elastic inference: none.



Create notebook instance

Amazon SageMaker provides pre-built fully managed notebook instances that run Jupyter notebooks. The notebook instances include example code for common model training and hosting exercises. [Learn more](#)

Notebook instance settings

Notebook instance name

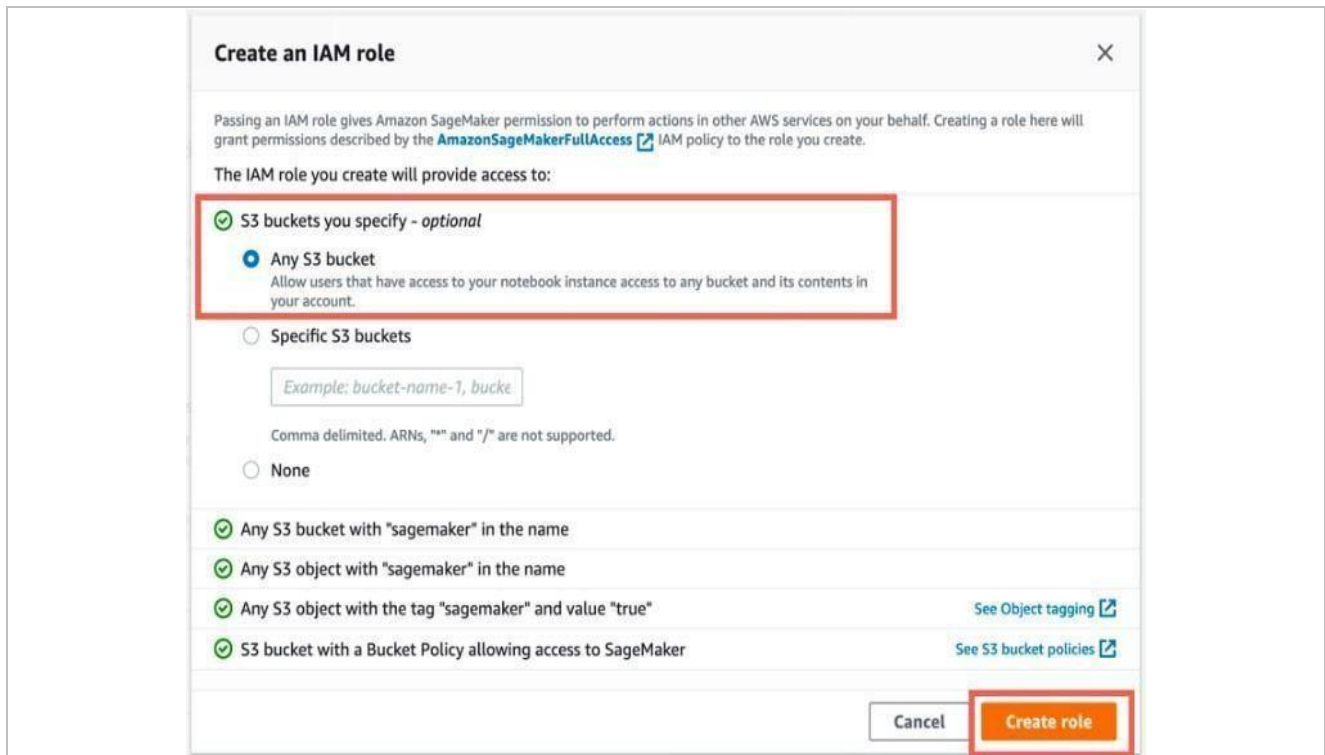
Maximum of 63 alphanumeric characters. Can include hyphens (-), but not spaces. Must be unique within your account in an AWS Region.

Notebook instance type

Elastic Inference [Learn more](#)

► Additional configuration

- d. In the Permissions and encryption section, for IAM role choose Create a new role, then in the Create an IAM role dialog select Any S3 bucket and choose Create role.



Create an IAM role

Passing an IAM role gives Amazon SageMaker permission to perform actions in other AWS services on your behalf. Creating a role here will grant permissions described by the [AmazonSageMakerFullAccess](#) IAM policy to the role you create.

The IAM role you create will provide access to:

- ☒ S3 buckets you specify - optional
 - ☒ Any S3 bucket
Allow users that have access to your notebook instance access to any bucket and its contents in your account.
 - ☐ Specific S3 buckets

Comma delimited. ARNs, "" and "/" are not supported.
 - ☐ None

- ☒ Any S3 bucket with "sagemaker" in the name
- ☒ Any S3 object with "sagemaker" in the name
- ☒ Any S3 object with the tag "sagemaker" and value "true" [See Object tagging](#)
- ☒ S3 bucket with a Bucket Policy allowing access to SageMaker [See S3 bucket policies](#)

*Amazon SageMaker creates the AmazonSageMaker-ExecutionRole-*** role*

Permissions and encryption

IAM role
Notebook instances require permissions to call other services including SageMaker and S3. Choose a role or let us create a role with the **AmazonSageMakerFullAccess** IAM policy attached.

AmazonSageMaker-ExecutionRole-20201204T135178

✔ **Success! You created an IAM role.**
AmazonSageMaker-ExecutionRole-20201204T135178

Root access - optional

☒ **Enable** - Give users root access to the notebook

☐ **Disable** - Don't give users root access to the notebook
Lifecycle configurations always have root access

Encryption key - optional
Encrypt your notebook data. Choose an existing KMS key or enter a key's ARN.

No Custom Encryption

- e. Keep the default settings for the remaining options and choose Create notebook instance. The new instance is shown with a status of Pending.

Amazon SageMaker > Notebook Instances

Notebook instances Create notebook instance

Search notebook instances

Name	Instance	Creation time	Status	Actions
SageMaker-Tutorial	ml.t2.medium	Jan 09, 2021 00:28 UTC	Pending	-

The notebook is ready when the status changes to InService

Amazon SageMaker > Notebook Instances

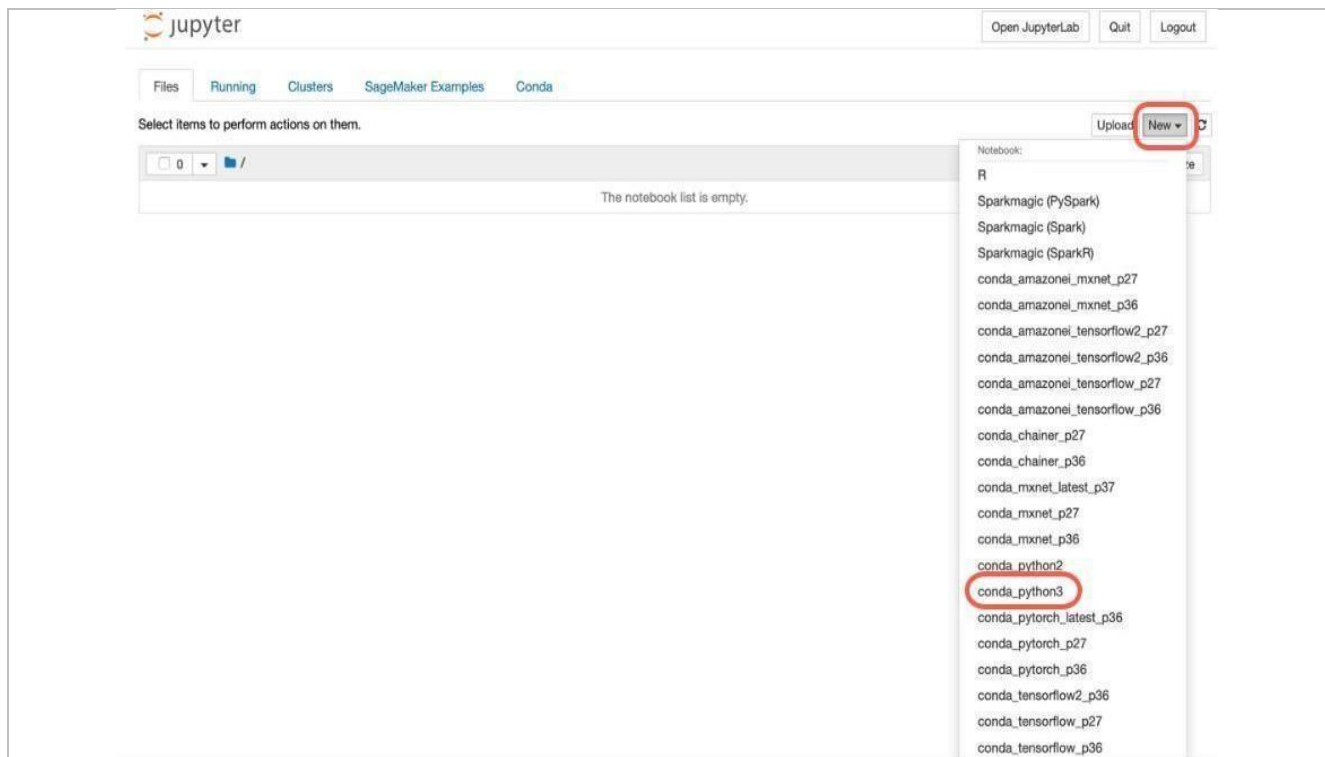
Notebook instances Create notebook instance

Search notebook instances

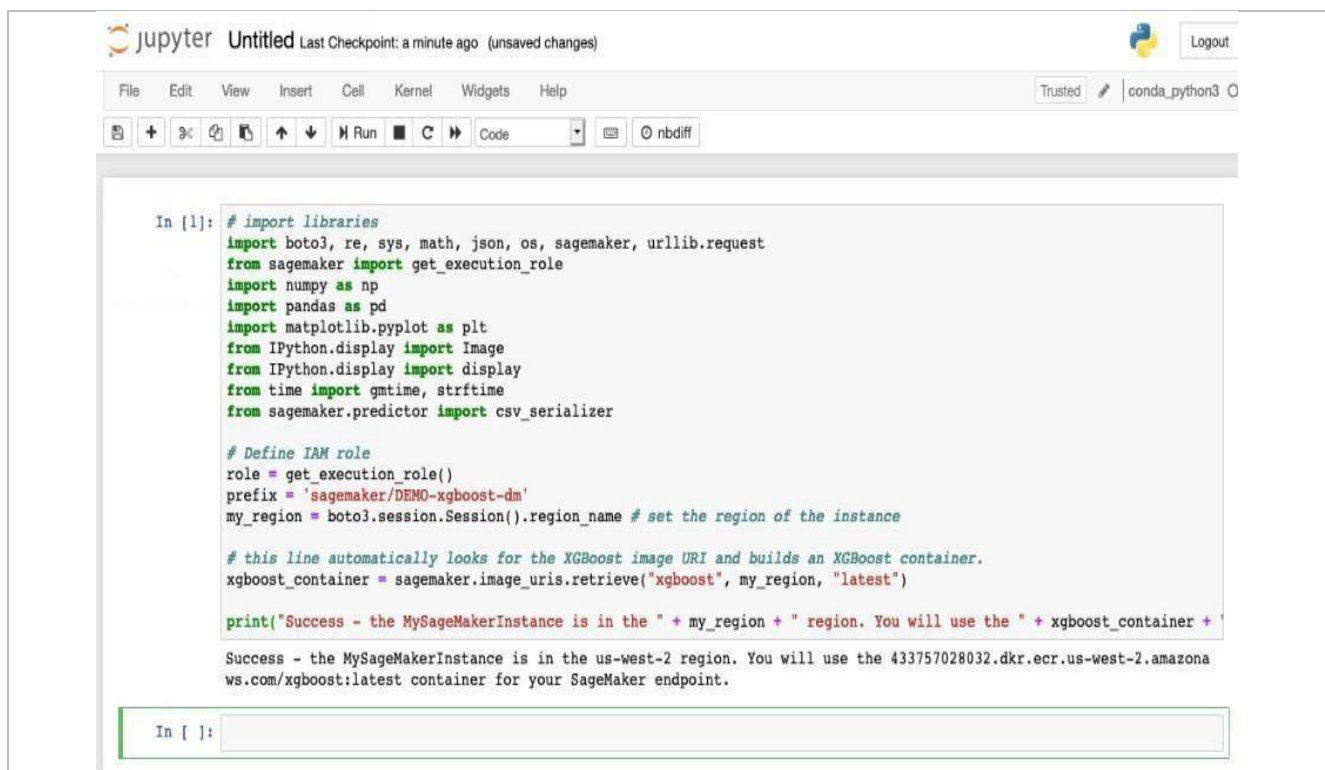
Name	Instance	Creation time	Status	Actions
SageMaker-Tutorial	ml.t2.medium	Jan 09, 2021 00:28 UTC	InService	Open Jupyter Open JupyterLab

Step 2 — Prepare the Data

- After the SageMaker-Tutorial notebook instance status changes to InService, choose Open Jupyter.
- In Jupyter, choose New, then choose conda_python3.



c. In a new code cell, paste the import/setup code and choose Run.



Continue running each of the remaining code sections — create bucket, download data, split data, train model, deploy, evaluate, clean up — in new cells, in order, as shown in the Program below.

PROGRAM

Import Libraries and IAM Role Setup

```
import boto3, re, sys, math, json, os, sagemaker, urllib.request
from sagemaker import get_execution_role
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import Image
from IPython.display import display
from time import gmtime, strftime
```

Define IAM role

```
role = get_execution_role()
prefix = 'sagemaker/DEMO-xgboost-dm'
my_region = boto3.session.Session().region_name # region of the instance
```

Automatically look up the XGBoost image URI and build the container

```
xgboost_container = sagemaker.image_uris.retrieve("xgboost", my_region, "latest")
print("Success - the MySageMakerInstance is in the " + my_region +
      " region. You will use the " + xgboost_container +
      " container for your SageMaker endpoint.")
```

Create the S3 Bucket (unique name, auto-generated)

```
account_id = boto3.client('sts').get_caller_identity().get('Account')
bucket_name = 'sagemaker-demo-xgboost-{}-{}'.format(
    account_id, strftime('%Y%m%d%H%M%S', gmtime()))

s3 = boto3.resource('s3')
try:
    if my_region == 'us-east-1':
        s3.create_bucket(Bucket=bucket_name)
    else:
        s3.create_bucket(
            Bucket=bucket_name,
            CreateBucketConfiguration={'LocationConstraint': my_region})
    print('S3 bucket created successfully: ' + bucket_name)
except Exception as e:
    print('S3 error: ', e)
```

Download and Load the Dataset

```
try:
    urllib.request.urlretrieve(
        "https://d1.awsstatic.com/tmt/build-train-deploy-machine-learning-model-sagemaker/"
        "bank_clean.27f01fbddf43271788427f3682996ae29ceca05d.csv",
```

```

        "bank_clean.csv")
    print('Success: downloaded bank_clean.csv.')
except Exception as e:
    print('Data load error: ', e)

try:
    model_data = pd.read_csv('./bank_clean.csv', index_col=0)
    print('Success: Data loaded into dataframe.')
except Exception as e:
    print('Data load error: ', e)

```

Shuffle and Split the Data (train 70% / test 30%)

```

train_data, test_data = np.split(
    model_data.sample(frac=1, random_state=1729),
    [int(0.7 * len(model_data))])
print(train_data.shape, test_data.shape)

```

Train the XGBoost Model

```

pd.concat([train_data['y_yes'],
           train_data.drop(['y_no', 'y_yes'], axis=1)],
          axis=1).to_csv('train.csv', index=False, header=False)

boto3.Session().resource('s3').Bucket(bucket_name).Object(
    os.path.join(prefix, 'train/train.csv')).upload_file('train.csv')

s3_input_train = sagemaker.inputs.TrainingInput(
    s3_data='s3://{}/{}/train'.format(bucket_name, prefix),
    content_type='csv')

sess = sagemaker.Session()
xgb = sagemaker.estimator.Estimator(
    xgboost_container, role, instance_count=1,
    instance_type='ml.m4.xlarge',
    output_path='s3://{}/{}/output'.format(bucket_name, prefix),
    sagemaker_session=sess)

xgb.set_hyperparameters(
    max_depth=5, eta=0.2, gamma=4, min_child_weight=6,
    subsample=0.8, verbosity=0, objective='binary:logistic',
    num_round=100)

xgb.fit({'train': s3_input_train})

```

Deploy the Model and Run Predictions

```
xgb_predictor = xgb.deploy(initial_instance_count=1,
                           instance_type='ml.m4.xlarge')

from sagemaker.serializers import CSVSerializer
test_data_array = test_data.drop(['y_no', 'y_yes'], axis=1).values
xgb_predictor.serializer = CSVSerializer()
predictions = xgb_predictor.predict(test_data_array).decode('utf-8')
```

```
predictions_array = np.array(
    predictions.strip().split(','), dtype='float32')
print(predictions_array.shape)
```

Evaluate Model Performance

```
cm = pd.crosstab(index=test_data['y_yes'],
                 columns=np.round(predictions_array),
                 rownames=['Observed'], colnames=['Predicted'])
tn = cm.iloc[0, 0]; fn = cm.iloc[1, 0]
tp = cm.iloc[1, 1]; fp = cm.iloc[0, 1]
p = (tp + tn) / (tp + tn + fp + fn) * 100

print("\n{0:<20}{1:<4.1f}%\n".format("Overall Classification Rate: ", p))
print("{0:<15}{1:<15}{2:>8}".format("Predicted", "No Purchase", "Purchase"))
print("Observed")
print("{0:<15}{1:<2.0f}% ({2:<}){3:>6.0f}% ({4:<})".format(
    "No Purchase", tn / (tn + fn) * 100, tn, fp / (tp + fp) * 100, fp))
print("{0:<16}{1:<1.0f}% ({2:<}){3:>7.0f}% ({4:<}) \n".format(
    "Purchase", fn / (tn + fn) * 100, fn, tp / (tp + fp) * 100, tp))
```

Clean Up Resources

```
xgb_predictor.delete_endpoint(delete_endpoint_config=True)
bucket_to_delete = boto3.resource('s3').Bucket(bucket_name)
bucket_to_delete.objects.all().delete()
```

Then, from the SageMaker console:

- # 1. Open Notebook instances
- # 2. Select the notebook instance -> Actions -> Stop
- # 3. Once Status changes to 'Stopped' -> Actions -> Delete -> Delete

OUTPUT:

Success - the MySageMakerInstance is in the ap-south-1 region. We will use the xgboost container for your SageMaker endpoint.

S3 bucket created successfully: sagemaker-demo-xgboost-<account-id>-<timestamp>

Success: downloaded bank_clean.csv. Success: Data loaded into dataframe.

(28831, 59) (12357, 59) — shapes of train_data and test_data

(12357,) — shape of predictions_array

Observed \ Predicted	No Purchase	Purchase
No Purchase	~90%	~10%
Purchase	~35%	~65%

Confusion matrix — overall classification rate approx. 90%. Exact figures vary by run since training data is shuffled randomly.

RESULT

The Amazon SageMaker toolkit was studied and successfully implemented.